项目 10 手势识别

1、 手势识别原理

1.1、 Mediapipe 库使用

MediaPipe 是一款由 Google Research 开发并开源的多媒体机器学习模型应用框架，可以直接调用其 API 完成目标检测、人脸检测以及关键点检测等。

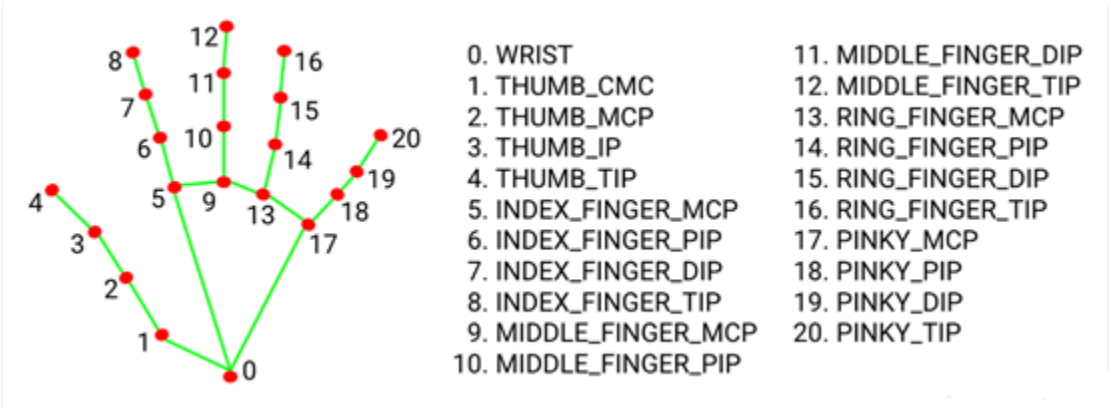


图 1-1 手部关键点

在 Ubuntu 系统中打开终端，输入 MediaPipe 的安装指令。

```
pip install mediapipe
```

1.2、 关键函数解释

1.2.1、 mediapipe.solutions.hands.Hands(): 手部关键点检测方法

参数解析：

static_image_mode：默认为 False，将输入图像视为视频流。它将尝试在第一个输入图像中检测手，并在成功检测后进一步定位手的坐标。在随后的图像中，一旦检测到所有 max_num_hands 手并定位了相应的手的坐标，它就会跟踪这些坐标，而不会调用另一个检测，直到它失去对任何一只手的跟踪。这减少了延迟，非常适合处理视频帧。如果设置为 True，则在每个输入图像上运行手部检测，用于处理一批静态的、可能不相关的图像。

max_num_hands： 最多检测几只手，默认为 2。

min_detection_confidence： 手部检测模型的最小置信值（0-1 之间），超过阈值则检测成功。默认为 0.5。

min_tracking_confidence： 坐标跟踪模型的最小置信值（0-1 之间），用于将手部坐标视为成功跟踪，不成功则在下一个输入图像上自动调用手部检测。将其设置为更高的值可以提高解决方案的稳健性，但代价是更高的延迟。如果 static_image_mode 为真，则忽略这个参数，手部检测将在每个图像上运行，默认为 0.5。

返回值：

MULTI_HAND_LANDMARKS：被检测/跟踪的手的集合，其中每只手被表示为 21 个手部地标的列表，每个地标由 x，y，z 组成。x

和 y 分别由图像的宽度和高度归一化为[0,1]。Z 表示地标深度。

MULTI_HANDEDNESS： 被检测/追踪的手是左手还是右手的集合。每只手由 label(标签)和 score(分数)组成。 label 是 ‘Left’ 或 ‘Right’ 值的字符串。 score 是预测左右手的估计概率。

1.2.2、 mediapipe.solutions.drawing_utils.draw_landmarks： 绘制手部关键点的连线

参数解析：

image： 需要画图的原始图片

landmark_list： 检测到的手部关键点坐标

connections： 连接线，需要把那些坐标连接起来

landmark_drawing_spec： 坐标的颜色，粗细

connection_drawing_spec： 连接线的粗细，颜色等

1.3、 例程：通过手部特征点识别特定手势

MediaPipe 库可以实现从简单到复杂手势的检测，受限于机器人头部摄像头的高度和广角的的限制，我们先从最简单的手势识别开始，通过检测手部特征点，识别当前视频数据中的伸直的手指总数 (0

到 10 之间)。

gesture_detection_process 手势识别函数：这是一个无限循环处理。

①初始化视频管道，从当前视频流中获取图像。

②使用 mediapipe 进行手势识别，当检测到当前手势的维持时间超过 5 秒，才发送手势识别结果到通信管道。

③在当前视频显示界面绘制识别结果。

以下代码中识别了三种手势数据，分别是数字 1、2、3（对应当前伸直的手指总数)，相应的动作指令是 Sit(坐下)、Heart(比爱心)、Hello(打招呼)。

```
    if result.multi_hand_landmarks:
                    for handLms in result.multi_hand_landmarks:
                        # 打印检测到的手势
                        print("Detected hand landmarks")
                        # 绘制手部关键点和骨骼
                        drawing_utils.draw_landmarks(image,     handLms,
mp_hands.HAND_CONNECTIONS)
                        #计算关键点的距离，用于判断手指是否伸直
```

```
                    hand_landmarks = handLms.landmark

                    p0_x = handLms.landmark[0].x

                    p0_y = handLms.landmark[0].y

                    p5_x = handLms.landmark[5].x

                    p5_y = handLms.landmark[5].y

                    distance_0_5    =    pow(p0_x-p5_x,2)+pow(p0_y-
p5_y,2)

                    base = distance_0_5 / 0.6

                    p4_x = handLms.landmark[4].x

                    p4_y = handLms.landmark[4].y

                    distance_5_4    =    pow(p5_x-p4_x,2)+pow(p5_y-
p4_y,2)

                    p8_x = handLms.landmark[8].x

                    p8_y = handLms.landmark[8].y

                    distance_0_8    =    pow(p0_x-p8_x,2)+pow(p0_y-
p8_y,2)

                    p12_x = handLms.landmark[12].x

                    p12_y = handLms.landmark[12].y
```

```
                    distance_0_12   =   pow(p0_x-p12_x,2)+pow(p0_y-
p12_y,2)
                    p16_x = handLms.landmark[16].x
                    p16_y = handLms.landmark[16].y
                    distance_0_16   =   pow(p0_x-p16_x,2)+pow(p0_y-
p16_y,2)
                    p20_x = handLms.landmark[20].x
                    p20_y = handLms.landmark[20].y
                    distance_0_20   =   pow(p0_x-p20_x,2)+pow(p0_y-
p20_y,2)
                    if distance_0_8 > base:
                        flag += 1
                    if distance_0_12 > base:
                        flag += 1
                    if distance_0_16 > base:
                        flag += 1
                    if distance_0_20 > base:
                        flag += 1
```

```
                    if distance_5_4 > base*0.3:
                        flag += 1
                    if flag>=10:
    # 检测手势 1、手势 2 和手势 3
                    if flag == 1 and gesture_last_sent !=
"gesture_one_detected":
                        if current_time - gesture_one_detected_time >
5:  #waiting for 5s
                            print("Gesture 1 detected")
                            pipe.send("gesture_one_detected")
                            gesture_last_sent =
"gesture_one_detected"
                            gesture_one_detected_time = 0
                        elif gesture_one_detected_time ==0:
                            gesture_one_detected_time = time.time()

                    if flag == 2 and gesture_last_sent !=
"gesture_two_detected":
```

```
                        if current_time - gesture_two_detected_time >
5:   #waiting for 5s
                            print("Gesture 2 detected")
                            pipe.send("gesture_two_detected")
                            gesture_last_sent                       =
"gesture_two_detected"
                            gesture_two_detected_time = 0
                        elif gesture_two_detected_time ==0:
                            gesture_two_detected_time = time.time()


                    if   flag   ==   3   and   gesture_last_sent   !=
"gesture_three_detected":
                        if                  current_time                -
gesture_three_detected_time > 5:   #waiting for 5s
                            print("Gesture 3 detected")
                            pipe.send("gesture_three_detected")
                            gesture_last_sent                       =
"gesture_three_detected"
```

```
                    gesture_three_detected_time = 0
                elif gesture_three_detected_time ==0:
                    gesture_three_detected_time = time.time()
```

2、 机器人运动控制

机器人运动控制类（SportModeTest）：

Stand、Sit、Hello、Heart 方法：分别用于控制机器人执行站立、坐下、打招呼和比心的动作。

```
class SportModeTest:
 def __init__(self) -> None:
     try:
         self.client = SportClient()
         self.client.SetTimeout(10.0)
         self.client.Init()
     except AttributeError as e:
         print(f"Error initializing SportClient: {e}")
 def Stand(self):
     try:
```

```
            self.client.StandUp()
            print("Stand up !!!")
        except Exception as e:
            print(f"Error executing RiseSit: {e}")
    def Sit(self):
        try:
            self.client.Sit()
            print("Sit down !!!")
        except Exception as e:
            print(f"Error executing Sit: {e}")
    def Hello(self):
        try:
            self.client.Hello()
            print("Hello !!!")
        except Exception as e:
            print(f"Error executing Hello: {e}")
    def Heart(self):
        try:
```

```
                self.client.Heart()
                print("Heart !!!")
            except Exception as e:
                print(f"Error executing Heart: {e}")
```

robot_control_process 函数：接收手势识别线程发送的手势指令，并控制机器人执行相应的动作。每执行完一个动作后，恢复到正常站立状态。

```
ChannelFactoryInitialize(0, network_interface)
    robot = SportModeTest()
    robot.Stand()
    dog_is_standing = True #Initialization State is standing
    while True:
        try:
            gesture = pipe.recv()  # 等待接收手势指令
            print(f"Received gesture: {gesture}")
            # 不需要冷却时间，立即执行动作
            if gesture == "gesture_one_detected":
                robot.Sit()
```

```
                dog_is_standing = False
            elif gesture == "gesture_two_detected":
                print("robot is showing Hello ")
                robot.Hello()
            elif gesture == "gesture_three_detected":
                print("robot is showing Heart ")
                robot.Heart()
            elif gesture == "stand":
                print("robot is standing ")
                robot.Stand()
                dog_is_standing = True
```

3、 多线程应用

在主函数中采用了多进程来分离手势识别和机器人控制的任务，以提高程序的响应性和效率。我们使用 multiprocessing 库创建两个子进程，一个用于手势检测，一个用于机器人控制。这两个进程通过管道进行通信，通过共享变量 last_action_time_shared，用于在子进程间共享最后动作的时间。

```
        # 捕获系统信号 (如 Ctrl+C) 并终止进程
        signal.signal(signal.SIGINT, signal_handler)
        signal.signal(signal.SIGTERM, signal_handler)
        parent_conn, child_conn = mp.Pipe()
        network_interface = sys.argv[1]  # 获取网络接口
        # 创建一个共享的冷却时间变量
        last_action_time_shared = Value('d', time.time())
        # 创建手势检测进程和机器人控制进程
            p1 = mp.Process(target=gesture_detection_process,
args=(parent_conn, network_interface, last_action_time_shared))
            p2 = mp.Process(target=robot_control_process,
args=(child_conn, network_interface, last_action_time_shared))
        p1.start()
        p2.start()
```

4、 完整代码

完整代码如下。

```
import time
```

```
import sys
import cv2
import numpy as np
import multiprocessing as mp
from multiprocessing import Value
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from unitree_sdk2py.go2.video.video_client import VideoClient
from unitree_sdk2py.go2.sport.sport_client import SportClient
import mediapipe as mp_mediapipe
import signal
#gesture lasting time
# 机器人运动控制类
class SportModeTest:
    def __init__(self) -> None:
        try:
            self.client = SportClient()
            self.client.SetTimeout(10.0)
            self.client.Init()
```

```
        except AttributeError as e:
            print(f"Error initializing SportClient: {e}")
    def Stand(self):
        try:
            self.client.StandUp()
            print("Stand up !!!")
        except Exception as e:
            print(f"Error executing RiseSit: {e}")
    def Sit(self):
        try:
            self.client.Sit()
            print("Sit down !!!")
        except Exception as e:
            print(f"Error executing Sit: {e}")
    def Hello(self):
        try:
            self.client.Hello()
            print("Hello !!!")
```

```
            except Exception as e:
                print(f"Error executing Hello: {e}")
        def Heart(self):
            try:
                self.client.Heart()
                print("Heart !!!")
            except Exception as e:
                print(f"Error executing Heart: {e}")
# 初始化视频客户端
def initialize_video_client(network_interface):
    ChannelFactoryInitialize(0, network_interface)
    client = VideoClient()
    client.SetTimeout(3.0)
    client.Init()
    return client
# 获取视频帧
def get_video_frame(client):
    code, data = client.GetImageSample()
```

```
        if code == 0:
            image_data = np.frombuffer(bytes(data), dtype=np.uint8)
            image = cv2.imdecode(image_data, cv2.IMREAD_COLOR)
            return image
        else:
            print("Get image sample error. code:", code)
            return None
    # 绘制检测区域
    def draw_detection_area(image, radius):
        height, width, _ = image.shape
        center = (width // 2, (height // 2) - 200)
        mask = np.zeros((height, width), dtype=np.uint8)
        cv2.circle(mask, center, radius, 255, thickness=-1)
        cv2.circle(image, center, radius, (0, 255, 0), 2)
        return mask
    # 手势检测线程
    def gesture_detection_process(pipe, network_interface,
last_action_time_shared):
```

```
        # 在子进程中重新初始化
        client = initialize_video_client(network_interface)
        mp_hands = mp_mediapipe.solutions.hands
        hands = mp_hands.Hands(min_detection_confidence=0.5,
min_tracking_confidence=0.5)
        drawing_utils = mp_mediapipe.solutions.drawing_utils
        radius = 800
        gesture_last_sent = None  # 保存上一次发送的手势指令
        gesture =
["none","one","two","three","four","five","six","seven","eight","nine","ten
"]
        gesture_one_detected_time = 0  # 手势 1 计时器
        gesture_two_detected_time = 0  # 手势 1 计时器
        gesture_three_detected_time = 0  # 手势 1 计时器
        while True:
            # 获取当前时间
            current_time = time.time()
            image = get_video_frame(client)
```

```
            if image is None:
                break
            mask = draw_detection_area(image, radius)
            imgRGB = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
            masked_image = cv2.bitwise_and(imgRGB, imgRGB,
mask=mask)
            result = hands.process(masked_image)
            flag = 0
            if result.multi_hand_landmarks:
                for handLms in result.multi_hand_landmarks:
                    # 打印检测到的手势
                    print("Detected hand landmarks")
                    # 绘制手部关键点和骨骼
                    drawing_utils.draw_landmarks(image, handLms,
mp_hands.HAND_CONNECTIONS)
                    #计算关键点的距离，用于判断手指是否伸直
                    hand_landmarks = handLms.landmark
```

```
p0_x = handLms.landmark[0].x

p0_y = handLms.landmark[0].y

p5_x = handLms.landmark[5].x

p5_y = handLms.landmark[5].y

distance_0_5 = pow(p0_x-p5_x,2)+pow(p0_y-p5_y,2)

base = distance_0_5 / 0.6



p4_x = handLms.landmark[4].x

p4_y = handLms.landmark[4].y

distance_5_4 = pow(p5_x-p4_x,2)+pow(p5_y-p4_y,2)



p8_x = handLms.landmark[8].x

p8_y = handLms.landmark[8].y

distance_0_8 = pow(p0_x-p8_x,2)+pow(p0_y-p8_y,2)



p12_x = handLms.landmark[12].x

p12_y = handLms.landmark[12].y

    distance_0_12  =  pow(p0_x-p12_x,2)+pow(p0_y-
```

```
p12_y,2)
                    p16_x = handLms.landmark[16].x
                    p16_y = handLms.landmark[16].y
                        distance_0_16  =  pow(p0_x-p16_x,2)+pow(p0_y-
p16_y,2)
                    p20_x = handLms.landmark[20].x
                    p20_y = handLms.landmark[20].y
                        distance_0_20  =  pow(p0_x-p20_x,2)+pow(p0_y-
p20_y,2)
                    if distance_0_8 > base:
                       flag += 1
                    if distance_0_12 > base:
                       flag += 1
                    if distance_0_16 > base:
                       flag += 1
                    if distance_0_20 > base:
                       flag += 1
                    if distance_5_4 > base*0.3:
```

```
                    flag += 1
                if flag>=10:
                    flag = 10
                # 检测手势 1、手势 2 和手势 3
                if flag == 1 and gesture_last_sent !=
"gesture_one_detected":
                    if current_time - gesture_one_detected_time >
5:  #waiting for 5s
                        print("Gesture 1 detected")
                        pipe.send("gesture_one_detected")
                        gesture_last_sent = "gesture_one_detected"
                        gesture_one_detected_time = 0
                    elif gesture_one_detected_time ==0:
                        gesture_one_detected_time = time.time()

            if flag == 2 and gesture_last_sent !=
"gesture_two_detected":
                    if current_time - gesture_two_detected_time >
```

```
5:  #waiting for 5s
                            print("Gesture 2 detected")
                            pipe.send("gesture_two_detected")
                            gesture_last_sent = "gesture_two_detected"
                            gesture_two_detected_time = 0
                        elif gesture_two_detected_time ==0:
                            gesture_two_detected_time = time.time()


            if flag == 3 and gesture_last_sent != "gesture_three_detected":
                            if current_time - gesture_three_detected_time >
5:  #waiting for 5s
                            print("Gesture 3 detected")
                            pipe.send("gesture_three_detected")
                            gesture_last_sent = "gesture_three_detected"
                            gesture_three_detected_time = 0
                        elif gesture_three_detected_time ==0:
                            gesture_three_detected_time = time.time()
            else:
```

```
                gesture_last_sent = "stand"  # 如果没有手势，let the go2
to stand
                pipe.send("stand")
                gesture_last_sent = "stand"
            # 显示视频流,手部骨骼
            cv2.putText(image,gesture[flag],(50,50),0,1.3,(0,0,255),3)
            cv2.imshow("front_camera", image)
            if cv2.waitKey(20) == 27:
                break
        cv2.destroyWindow("front_camera")
    # 机器人运动控制线程
    def robot_control_process(pipe, network_interface,
last_action_time_shared):
        # 在子进程中重新初始化
        ChannelFactoryInitialize(0, network_interface)
        robot = SportModeTest()
        robot.Stand()
        dog_is_standing = True #Initialization State is standing
```

```
while True:
    try:
        gesture = pipe.recv()  # 等待接收手势指令
        print(f"Received gesture: {gesture}")
        # 不需要冷却时间，立即执行动作
        if gesture == "gesture_one_detected":
            robot.Sit()
            dog_is_standing = False
        elif gesture == "gesture_two_detected":
            print("robot is showing Hello ")
            robot.Hello()
        elif gesture == "gesture_three_detected":
            print("robot is showing Heart ")
            robot.Heart()
        elif gesture == "stand":
            print("robot is standing ")
            robot.Stand()
            dog_is_standing = True
```

```
            except EOFError:
                break
    # 捕获信号处理，确保子进程能够优雅关闭
    def signal_handler(sig, frame):
        print("Signal received, terminating...")
        sys.exit(0)
    # 主函数
    def main():
        # 捕获系统信号 (如 Ctrl+C) 并终止进程
        signal.signal(signal.SIGINT, signal_handler)
        signal.signal(signal.SIGTERM, signal_handler)
        parent_conn, child_conn = mp.Pipe()
        network_interface = sys.argv[1]  # 获取网络接口
        # 创建一个共享的冷却时间变量
        last_action_time_shared = Value('d', time.time())
        # 创建手势检测进程和机器人控制进程
            p1 = mp.Process(target=gesture_detection_process,
args=(parent_conn, network_interface, last_action_time_shared))
```

```
                p2 = mp.Process(target=robot_control_process,
args=(child_conn, network_interface, last_action_time_shared))
        p1.start()
        p2.start()
        try:
            p1.join()
            p2.join()
        except KeyboardInterrupt:
            print("KeyboardInterrupt caught, terminating processes...")
        finally:
            # 确保所有子进程被终止
            p1.terminate()
            p2.terminate()
            p1.join()
            p2.join()
    if __name__ == "__main__":
        main()
```

5、 项目总结

通过本项目的学习，我们掌握了 Mediapipe 库的使用，通过手势识别来控制机器人执行不同的高级动作。下一个项目我们将延续数据识别结合运动控制的应用思路，让 Go2 机器人追踪目标物体运动。